Programming Assignment: Library Book Management System
Object-Oriented Programming

Week 6 Programming Assignment

Library Book Management System

Course: Programming Fundamentals

Topics Covered: Object-Oriented Programming, Classes (attributes, methods, __init__, __str__), and Inheritance (subclasses, super(), method overriding)

Prior Knowledge: Lists, dictionaries, modules, file I/O including CSV, exception handling, string operations, loops, functions, conditional logic, lists, tuples, strings, input, and print

---

How to Use This Assignment

This assignment has three tiers. Every student completes the Base Level. If you have prior programming experience, you are expected to also complete the section that matches your background:

- **Base Level —** Required for all students
- **Intermediate Level —** Required if you have programmed in Python or another language before
- **Advanced Level —** Required if you are comfortable with multiple languages or have significant programming experience

If you are unsure which tier applies to you, ask your instructor.

Note on structure: Each tier builds directly on the previous one. Your final submission should be a single .py file that contains all class definitions and a main program section. Intermediate students should have the Base Level class plus their two new subclasses. Advanced students should have all classes plus the Library class and a working interactive program.

Programming Assignment: Library Book Management System
Object-Oriented Programming

BASE LEVEL

Overview

In this assignment, you will build a Library Book Management System using object-oriented programming. At the Base Level, you will define a Book class that models a single library book as an object. Each Book object will know its own data (title, author, ISBN, year, and genre) and will be capable of performing actions on itself (being checked out, being returned, and reporting its status).

This is your first assignment using classes. The key shift from prior weeks is that instead of writing standalone functions that receive data as arguments, you will be writing methods — functions that belong to a class and always have access to the object's own data through the self parameter. Think of self as the object's way of referring to itself.

At the Base Level, you will define the Book class, create a list of Book objects, and interact with that list using the methods you write. You will not yet use inheritance — that is introduced at the Intermediate Level.

Requirements

Your Book class must include all of the following:

- An __init__ method that accepts and stores five parameters: title, author, isbn, year, and genre. It must also initialize two additional attributes without parameters: available (set to True) and borrower (set to None).
- A __str__ method that returns a single formatted string representing the book's key details. This method must use an f-string with at least one format specifier.
- A check_out method that accepts a patron_name parameter, checks whether the book is available, updates the object's state if it is, and returns a boolean.
- A return_book method that resets the book's availability and clears the borrower, and returns a descriptive confirmation string.
- A get_status method that returns a string describing whether the book is available or who has it checked out.

Your main program section must:

- Create a list of at least six Book objects using hardcoded starter data
- Display all books using their __str__ method
- Perform at least two check_out operations and display the result of each
- Perform at least one return_book operation and display the result
- Display a sorted view of the collection (sorted by title using sorted() with a key)
- Display a filtered view showing only available books

Programming Assignment: Library Book Management System

Object-Oriented Programming

Problem to Solve

The Book Class

Define the class using the class keyword at the top of your file, before the main program. All five methods described below must be defined inside the class body, indented under it.

__init__(self, title, author, isbn, year, genre)

The initializer method. Every Book object is created by calling this method, even though you do not call it directly — Python calls it automatically when you write Book(...). Store each parameter as an instance attribute using self.title = title, and so on for each parameter. Then set two additional attributes that are not passed in as parameters:

- self.available = True — every new book starts out available to be borrowed
- self.borrower = None — no one has it checked out yet

__str__(self)

This special method controls what Python prints when you call print(book) or str(book). It must return a string — not print one. The string should include the book's ISBN, title, author, year, genre, and its current status (call self.get_status() inside this method to get the status string). Format the output so it is readable and columns are reasonably aligned using f-string format specifiers.

check_out(self, patron_name)

This method attempts to check out the book to a patron. If self.available is True, update self.available to False, set self.borrower to patron_name, and return True. If self.available is already False, do not change anything and return False. This method must not print anything — the caller is responsible for deciding what to display based on the return value.

return_book(self)

This method processes a book being returned. Set self.available back to True and set self.borrower back to None. Return a confirmation string such as f"'{self.title}' has been returned and is now available." Do not print inside this method.

get_status(self)

This method returns a short status string. If self.available is True, return "Available". If not, return f"Checked out to {self.borrower}". This method is called by __str__ and can also be called directly by the main program.

Starter Data

In your main program section, create a list named collection containing at least six Book objects. Choose any books you like. Here is an example of how to create one:

Programming Assignment: Library Book Management System

Object-Oriented Programming

```
collection = [
    Book("The Great Gatsby","F. Scott Fitzgerald","978-0743273565",
1925,"Classic Fiction"),
    Book("Educated","Tara Westover","978-0399590504",2018,"Memoir"),
    Book("Dune","Frank Herbert","978-0441013593",1965,"Science
Fiction"),
    # ... add at least three more ...
]
```

Main Program Steps

After defining the collection list, write the following sequence of steps:

1. Display the heading "=== Full Collection ===" and then loop through the collection, printing each book using print(book). Because you defined __str__, this will call your formatting method automatically.
2. Check out the first two books in the list to different patron names by calling .check_out() on each. Use the return value to print either a success message (e.g., "'Dune' checked out to Jordan.") or an already-checked-out message.
3. Attempt to check out the first book again (it is now unavailable). Confirm that check_out() returns False and display an appropriate message.
4. Return the second checked-out book by calling .return_book() and printing the string it returns.
5. Display the collection sorted by title: use sorted(collection, key=lambda b: b.title) and print each book in the sorted result.
6. Display only the available books: build a new list using a list comprehension that filters for books where book.available is True, then print each one.

Example Output (partial)

```
=== Full Collection ===
978-0441013593 Dune by Frank Herbert:
      1965  Sci-Fi      Available

978-0743273565 The Great Gatsby by F. Scott Fitzgerald:
      1925  Classic     Available
...

'Dune' checked out to Jordan.
'The Great Gatsby' checked out to Priya.
'Dune' is already checked out.

'The Great Gatsby' has been returned and is now available.

=== Available Books ===
978-0743273565  The Great Gatsby  ...  Available
978-0399590504  Educated          ...  Available
```

Programming Assignment: Library Book Management System

Object-Oriented Programming

Testing & Quality

Before submitting, verify your program with the test cases below. Record actual output in a comment block at the top of your file.

Test Case	What to Verify
Print a newly created Book	__str__ returns a correctly formatted string including all five original fields plus status "Available"
check_out() on an available book	Returns True; book.available becomes False; book.borrower is set to the patron name
check_out() on an already checked-out book	Returns False; book.available and book.borrower are unchanged
return_book() on a checked-out book	Returns a confirmation string; book.available becomes True; book.borrower becomes None
Sorted display	Books appear in alphabetical order by title; sort does not permanently reorder the original list
Available filter	Only books where available == True appear in the filtered list after check-outs have been performed

Code quality checklist:

- ☐ The Book class is defined before the main program, not inside it
- ☐ __str__ returns a string — it does not call print()
- ☐ check_out() and return_book() do not print anything — the main program handles all output
- ☐ self is the first parameter of every method inside the class
- ☐ Instance attributes are set with self.attribute_name = value inside __init__
- ☐ Each method has a one-line comment describing what it does
- ☐ The available filter uses a list comprehension, not a for loop with manual appending

Programming Assignment: Library Book Management System
Object-Oriented Programming

INTERMEDIATE LEVEL

Complete the Base Level first, then add the classes described below to your file.

Overview

You will extend the Book Management System by introducing inheritance. In the real world, a library holds more than just traditional print books — it also lends eBooks, audiobooks, and keeps reference books that cannot be borrowed. Each of these is a kind of book, which means it shares the core properties of your Book class but also has its own unique attributes and behaviors.

Inheritance lets you express this relationship in code. Instead of rewriting the title, author, ISBN, year, and genre logic from scratch, your new classes will inherit all of it from Book and only add or override what is different. This is one of the central ideas of object-oriented programming: model what things have in common, then specialize for what makes each type unique.

At this level, you will define three subclasses — EBook, AudioBook, and ReferenceBook — and then demonstrate that a mixed collection of all four types can be managed through the same interface.

Requirements

In addition to everything from the Base Level, your program must:

- Define three subclasses that each explicitly inherit from Book using the class SubclassName(Book): syntax
- Call super().__init__(...) inside each subclass __init__ to initialize the inherited attributes
- Add at least one unique instance attribute per subclass, set in the subclass __init__
- Add at least one unique method per subclass that is not present in Book
- Override __str__ in each subclass by calling super().__str__() and appending subclass-specific information
- Override at least one other method (not __str__) in at least one subclass to change its behavior
- Demonstrate polymorphism: create a mixed list of all four types and call the same method on each object in a loop, showing that each type responds according to its own implementation

Problem to Solve

EBook(Book)

An EBook is a digital version of a book that can be accessed by multiple patrons simultaneously. Define EBook as a subclass of Book:

Programming Assignment: Library Book Management System

Object-Oriented Programming

__init__(self, title, author, isbn, year, genre, file_format, file_size_mb)

Call super().__init__(title, author, isbn, year, genre) to set the inherited attributes. Then set two new instance attributes: self.file_format (a string such as "PDF" or "EPUB") and self.file_size_mb (a float). Also set self.active_downloads = 0 — this will track how many patrons are currently accessing the file.

Override check_out(self, patron_name)

Because a digital file can be accessed by many people at once, an EBook is never unavailable. Override check_out() so that it always increments self.active_downloads by 1 and always returns True, regardless of any prior checkouts. Do not set self.available to False and do not set self.borrower — those fields are not meaningful for a digital resource.

Override get_status(self)

Return a string that reports the current download count: f"Digital — {self.active_downloads} active download(s)". If active_downloads is 0, return "Digital — available for download".

Override __str__(self)

Call super().__str__() to get the inherited display string, then concatenate a second line (using a newline character) that shows the file format and size: f" [Format: {self.file_format} | {self.file_size_mb} MB]". Return the combined string.

get_download_info(self)

A new method (not in Book). Return a formatted string summarizing the digital file details: f"Format: {self.file_format} | Size: {self.file_size_mb} MB | Downloads: {self.active_downloads}".

AudioBook(Book)

An AudioBook is a narrated version of a book. It behaves exactly like a print Book for checkout purposes but adds narrator and listening-time information.

__init__(self, title, author, isbn, year, genre, narrator, duration_hours)

Call super().__init__(title, author, isbn, year, genre). Then set self.narrator and self.duration_hours as new instance attributes.

Override __str__(self)

Call super().__str__() and append a second line: f" [Narrator: {self.narrator} | Duration: {self.duration_hours} hrs]".

get_listening_info(self)

A new method. Return f"Narrated by {self.narrator}. Listening time: {self.duration_hours} hours.". If duration_hours is greater than 10, also append " (Long listen — plan accordingly!)".

Programming Assignment: Library Book Management System

Object-Oriented Programming

ReferenceBook(Book)

A ReferenceBook (encyclopedia, atlas, style guide, etc.) is always kept in the library and cannot be checked out. It can be used on-site only.

__init__(self, title, author, isbn, year, genre, edition)

Call super().__init__(title, author, isbn, year, genre). Set self.edition as a new integer attribute. Also set self.available = False immediately after calling super().__init__() to permanently mark the book as non-circulatable.

Override check_out(self, patron_name)

Always return False without modifying any attribute. Print the message "Reference books are for in-library use only and cannot be checked out." Note: this is one of the rare cases where printing inside a method is appropriate because the behavior is a hard constraint, not an outcome the caller needs to decide how to display.

Override return_book(self)

Always return the string "Reference books do not need to be returned." Do not change any attribute.

Override get_status(self)

Always return f"In-Library Use Only (Edition {self.edition})".

Override __str__(self)

Call super().__str__() and append a line: f" [Reference — Edition {self.edition} — In-Library Use Only]".

Demonstrating Polymorphism

In your main program, after the Base Level section, create a new list called mixed_collection that contains at least two objects of each type (Book, EBook, AudioBook, ReferenceBook). Then write a for loop that iterates over the list and:

- Calls print(item) on each object — demonstrating that __str__ produces different output for each type
- Calls item.check_out("Test Patron") on each object and prints whether it returned True or False, demonstrating that the same method call produces type-appropriate behavior
- Calls print(item.get_status()) on each object after the check_out attempt

This section should show clearly that the same three method calls produce different behavior depending on the actual type of each object in the list.

Programming Assignment: Library Book Management System

Object-Oriented Programming

Example Polymorphism Output (partial)

```
--- Polymorphism Demonstration ---

978-0743273565  The Great Gatsby  ...  Available
  check_out result: True
  status: Checked out to Test Patron

978-1234567890  Clean Code (EBook)  ...  Digital — available for
download
  [Format: EPUB | 1.2 MB]
  check_out result: True
  status: Digital — 1 active download(s)

978-0000000001  Chicago Manual (Ref.)  ...  In-Library Use Only
(Edition 17)
  [Reference — Edition 17 — In-Library Use Only]
  Reference books are for in-library use only and cannot be checked
out.
  check_out result: False
  status: In-Library Use Only (Edition 17)
```

Testing & Quality

Test your program with the following scenarios in addition to the Base Level test cases:

Test Scenario	What to Verify
Check out an EBook twice	active_downloads increments to 2; available remains True; get_status() reflects both downloads
Check out a ReferenceBook	Returns False; message is printed; no attribute changes
return_book() on a ReferenceBook	Returns the correct string; no attribute changes
print() on each type	Each __str__ output includes the inherited Book line plus the subclass-specific second line
get_listening_info() on a long AudioBook (>10 hrs)	Long-listen note appended to the return string
super().__init__() called in each subclass	All Book attributes (title, author, isbn, year, genre, available, borrower) are accessible on every subclass instance

Code quality checklist (in addition to Base):

- □ Each subclass header uses class SubclassName(Book): syntax
- □ super().__init__(...) is called as the first line of each subclass __init__
- □ Each subclass __str__ calls super().__str__() rather than re-implementing the Book display logic

Programming Assignment: Library Book Management System

Object-Oriented Programming

- □ No Book attribute (title, author, isbn, etc.) is redefined in a subclass __init__ — they are inherited
- □ The polymorphism loop uses the same method names on all types and does not use isinstance() or type checks to branch
- □ ReferenceBook.available is set to False inside __init__ after calling super().__init__()

Programming Assignment: Library Book Management System
Object-Oriented Programming

ADVANCED LEVEL

Complete both the Base and Intermediate levels first, then add the Library class and interactive program described below.

Overview

You will build a Library class that acts as a manager for the entire book collection. Rather than working with a bare list in your main program, the Library object will encapsulate the collection and expose well-defined methods for every operation: adding and removing books, searching by author or genre, generating reports, and saving or loading the full catalog from a CSV file.

This level also introduces a practical technique for working with class hierarchies and files: using the isinstance() function to check what type an object actually is, so that subclass-specific attributes can be serialized to and reconstructed from CSV correctly. You will also build an interactive command-line menu loop that lets a user operate the library during a session.

Requirements

In addition to everything from the Base and Intermediate levels, your final program must:

- Define a Library class that is not a subclass of Book — it is an independent class that manages Book objects
- Use isinstance(obj, ClassName) in at least one method to check an object's type
- Save the full collection to a CSV file using the csv module, with a type column that records each book's class name
- Load the collection from the same CSV file, using the type column to instantiate the correct class for each row
- Wrap all file operations in try/except blocks with appropriate except clauses
- Build and return a multi-line report string from a method (do not print directly inside the method)
- Include an interactive while loop menu in the main program that lets the user perform operations until they choose to quit

Problem to Solve

The Library Class

__init__(self, name)

Store self.name as the library's display name. Initialize self.collection as an empty list. This list will hold all Book (and subclass) objects.

add_book(self, book)

Append the book object to self.collection. Return nothing.

remove_book(self, isbn)

Programming Assignment: Library Book Management System

Object-Oriented Programming

Search self.collection for a book whose isbn attribute matches the given isbn string. If found, remove it from the list and return True. If not found, return False. The ISBN comparison should be exact (no case conversion needed for ISBNs).

find_by_isbn(self, isbn)

Return the first Book object in self.collection whose isbn matches the argument, or None if no match is found.

find_by_author(self, author)

Return a list of all Book objects whose author attribute contains the given string (case-insensitive). Use .lower() on both sides. Return an empty list if there are no matches.

find_by_genre(self, genre)

Return a list of all Book objects whose genre attribute matches the given string (case-insensitive). Return an empty list if there are no matches.

get_available_books(self)

Return a list of all books in self.collection where book.available is True.

get_checked_out_books(self)

Return a list of all books where book.available is False. Note: an EBook's available is always True, so EBooks will never appear in this list by design.

generate_report(self)

Build and return a formatted multi-line string (do not print inside this method). The report must include:

- Library name and total number of books in the collection
- Count of available books and count of checked-out books
- A breakdown by type using isinstance(): how many are Book (base only), EBook, AudioBook, and ReferenceBook
- A section listing each checked-out book by title and borrower name
- Use string multiplication for separator lines and f-string format specifiers for alignment

save_to_csv(self, filename)

Write the full collection to a CSV file. Use the following column layout:

```
type, title, author, isbn, year, genre, available, borrower, extra1,
extra2
```

Programming Assignment: Library Book Management System

Object-Oriented Programming

The type column holds the class name as a string: "Book", "EBook", "AudioBook", or "ReferenceBook". The extra1 and extra2 columns hold subclass-specific data:

Type	extra1	extra2
Book	(empty string)	(empty string)
EBook	file_format	file_size_mb
AudioBook	narrator	duration_hours
ReferenceBook	edition	(empty string)

Use isinstance() to determine which type each object is before writing its row. Write the header row first. Wrap everything in a try/except block that catches Exception and prints a user-friendly save error message. Return the number of records written (not counting the header).

load_from_csv(self, filename)

Read the CSV file and reconstruct the collection. Clear self.collection before loading so that duplicate entries are not created on repeated loads. Use the type column to determine which class to instantiate for each row:

- "Book" → Book(title, author, isbn, int(year), genre)
- "EBook" → EBook(title, author, isbn, int(year), genre, extra1, float(extra2))
- "AudioBook" → AudioBook(title, author, isbn, int(year), genre, extra1, float(extra2))
- "ReferenceBook" → ReferenceBook(title, author, isbn, int(year), genre, int(extra1))

After constructing each object, restore the available and borrower fields from the CSV: set book.available = (available_str == "True") and book.borrower = None if the borrower field is empty, or the borrower string if it is not.

Wrap the entire method in a try/except block. Catch FileNotFoundError separately from ValueError and from a general Exception fallback. On FileNotFoundError, print an informational message (not an error) and return without modifying the collection. Return the number of records loaded.

Interactive Menu

In the main program section, create a Library object, load from "library.csv" (which may not exist on the first run), and then enter a while loop that presents the following menu until the user enters "q" or "quit":

```
[1] Display all books        [2] Add a book
[3] Check out a book         [4] Return a book
[5] Search by author         [6] Search by genre
[7] Display available books  [8] Generate report
[q] Save and quit
```

For option [2], ask the user for the book type first (Book / EBook / AudioBook / ReferenceBook), then prompt for the appropriate fields based on the type. Use conditionals to determine which class to instantiate. For options [3] and [4], ask for an ISBN, use find_by_isbn() to locate the book, and report whether the operation succeeded. For options [5] and [6], display the results using a for loop that prints each matching book. When the user quits, call save_to_csv() and print the confirmation.

Example Report Output

```
======================================
   LIBRARY REPORT: Riverside Public
======================================
Total books in collection : 9
Available                 : 6
Checked out               : 2

--- Collection by Type ---
Print Books    : 4
eBooks         : 2
Audiobooks     : 2
Reference      : 1

--- Currently Checked Out ---
  'Dune'              →  Jordan
  'Educated'          →  Priya
======================================
```

Testing & Quality

In addition to all previous test cases:

Test Scenario	What to Verify
Save 9 books to CSV, then reload	All 9 books are restored with correct types, fields, and available/borrower state; no duplicates
Load a CSV that has a checked-out Book	book.available is False and book.borrower holds the correct patron name after loading
Load a non-existent CSV file	FileNotFoundError is caught; informational (not error) message is printed; collection is unchanged
generate_report() with one EBook	EBook counted in the eBook row; not counted in checked-out section even if active_downloads > 0
remove_book() with a valid ISBN	Returns True; book no longer appears in collection
remove_book() with a non-existent ISBN	Returns False; collection is unchanged
Menu option [3] on a ReferenceBook	check_out() returns False; the reference-book message is printed; no state change

Programming Assignment: Library Book Management System

Object-Oriented Programming

Test Scenario	What to Verify
Add an AudioBook via the menu, then save and reload	narrator and duration_hours are correctly serialized and reconstructed from the extra1/extra2 columns

Code quality checklist (in addition to Intermediate):

- □ Library is defined as an independent class — class Library: with no parent class
- □ save_to_csv() uses isinstance() to determine each book's type before writing its row
- □ load_from_csv() restores available and borrower correctly from the CSV string values
- □ generate_report() builds and returns a string — it does not print directly
- □ The interactive menu is a while loop that terminates cleanly when the user enters "q"
- □ All file operations are inside try/except blocks; FileNotFoundError is caught separately from ValueError
- □ The main program section is concise because all logic is in Library methods

---

Programming Assignment: Library Book Management System
Object-Oriented Programming

Grading Rubric and Category Breakdown

Category	Points	What Is Being Assessed
Correctness	35	Program runs without errors and produces accurate output for all required test cases at the submitted tier.
Class Design	20	Classes are correctly defined with __init__, __str__, and required methods; instance attributes are set correctly with self; methods return values rather than printing where required.
Inheritance & Polymorphism	20	Subclasses correctly inherit from Book; super().__init__() is called; methods are overridden appropriately; the polymorphism demonstration shows type-specific behavior from uniform method calls.
Integration of Prior Knowledge	15	File I/O with CSV, exception handling, list comprehensions, string formatting, loops, and functions from prior weeks are applied correctly at the tiers that require them.
Output Quality	5	Output is clearly labeled and readable; monetary or numeric values are formatted; __str__ output is consistently structured across all types.
Code Quality	5	Descriptive names; methods have single responsibilities; no logic duplicated across subclasses that could be inherited; comments present on each class and method. There exists a comment block at the top of the program identifying the program's author, date, description, and tier level attempted.
Python code submitted correctly to GitHub account.	5	Student submitted the link to the student's public repository hosting this assignment's programming solution.
Total Base Score:	100	The base assignment is eligible for the full 100 points.
Intermediate Level:	5	Full implementation of intermediate level requirements beyond the base level (or substantial attempt was made).
Advanced Level:	8	Full implementation of advanced level requirements beyond the base level (or substantial attempt was made).

Final scores may exceed 100 if intermediate or advanced work is completed successfully. Extra credit is awarded only when the base assignment is complete, and the extension work runs correctly.

***Questions**? Post in the course FAQ discussion board or visit office hours.*